

Pre-Approval Website-Verification Tool

Project brief for candidate AI engineers

Version: Draft 1 · **Audience:** external AI/software engineers being evaluated · **Owner:** the Agency Implementation Team

0. What this document is (and how to read it)

This is a **take-home / interview project**. We are hiring AI engineers, and we want to see how you take a real, messy business problem and turn it into a clean, working AI tool with a well-organized repository.

You are **not** expected to build a production system. You are expected to build a **basic working solution** that demonstrates the workflow below, and to explain your choices, your assumptions, and what production would require.

This brief is deliberately self-contained. You do **not** need any prior knowledge of the Agency, government disability programs, or the forms involved — everything you need is explained here in plain language. Read Sections 1–2 for context, Sections 3–7 for the actual requirements, and Sections 8–11 for what we'll grade.

1. Background (plain English — no prior knowledge assumed)

Who we are. Our client is a non-profit human-services agency in New York. Among many programs, we help people with developmental disabilities who are enrolled in a New York State program called **OPWDD Self-Direction**.

What "Self-Direction" means. Normally an agency decides what services a disabled person gets. In *Self-Direction*, the **participant** (the person receiving services) is in the driver's seat: they control a personal, government-funded **budget** and choose how to spend it — hiring their own staff, signing up for classes, buying equipment that helps them live more independently, and so on.

What "FI" means. Someone has to hold that budget, act as the legal employer of the participant's staff, keep the books, and make sure every dollar is spent according to the rules. That role is called the **Fiscal Intermediary (FI)**. the Agency is the FI for these participants. An **FI Coordinator** is the Agency staff member who manages a caseload of participants.

What "FI intake" means. *FI intake* is the onboarding process when a new participant joins the Agency's Self-Direction program — collecting their paperwork, setting up their record, and establishing their budget. **This project is not about intake itself**; the term is defined here only because it appears in our internal materials and is easy to confuse with the process this project is about, which is **pre-approval** (below).

What "pre-approval" means — the heart of this project. The participant's budget can only be spent on things that meet strict government (OPWDD / Medicaid) rules. So before a participant is allowed to spend

budget money on certain non-routine things — a community class, a gym membership, a household item, a coaching course, etc. — they must get **pre-approval**: written sign-off from the Agency 's internal **Pre-Approvals team** confirming the purchase is allowed under the rules.

Why the rules are strict — the reason this tool matters. OPWDD and Medicaid **audit** these purchases. If the Agency approves something that didn't qualify, the money can be clawed back and the Agency is on the hook. A recurring rule across categories is that the thing being paid for must be **genuinely public and legitimate** — for example, a class must be *open to the general public* and have *publicly published fees* (not a special price invented for disabled/OPWDD participants). The cleanest way to prove that at audit time is: **go to the provider's public website and show that the class, its schedule, and its published price are really there, open to everyone.**

The person who does this today. A member of the Pre-Approvals team (the **Pre-Approvals Reviewer**) receives each application and today does the website check **by hand**: opens the provider's website, hunts for the class/fee/schedule, takes screenshots as evidence, and records what they found against the checklist. This is slow, repetitive, and inconsistent — and it is exactly what we want an AI tool to assist with.

2. The problem we want solved

Pre-Approvals Reviewers spend hours manually visiting provider websites to verify that the classes, memberships, items, and programs on pre-approval applications are publicly available at the stated price — and capturing screenshot evidence for the audit file. We want an AI tool that does the reading, the website research, and the evidence-gathering for them, and hands back a clear, review-ready report.

The tool **assists** the reviewer. It does **not** approve or deny anything — a human always makes the final call (see §10).

3. What the tool must do (the core workflow)

Given a **completed pre-approval application** (one of the forms in §4), the tool should:

1. **Read the application** and extract the key request details (see §4 for the fields).
2. **Identify the request category** (Community Class, Membership, Coaching, HRI, OTPS, Transition Program, or Appeal) — usually obvious from which form it is.
3. **Identify the provider/vendor website URL or product link** from the form.
4. **Identify the specific thing requested** (class name, membership name, item, program) and any **fee/price** stated on the form.
5. **Select the correct checklist** for that category (see §5) and figure out **which checklist items are verifiable from a public website** vs. which are internal (the tool only checks the website-verifiable ones).

6. **Visit the website and gather evidence** for each website-verifiable item — is the class/item there? Is it open to the public? Are fees published? Does the published price match the form?
7. **Capture screenshots** of the evidence it finds.
8. **Produce a review-ready report** (see §6) with a per-item status: **Found / Not Found / Needs Review**, each backed by an evidence URL, screenshot, and note.
9. **Let the reviewer interact in plain language** to adjust the report, add notes, or re-run (see §7).

4. Inputs — the pre-approval forms

Seven blank form templates are provided with this brief (PDFs). A real application is one of these forms **filled in** (typically a scanned or exported PDF). The tool must support all seven and be **designed so new forms/checklists can be added later** without rewriting the core.

4.1 Fields common to (almost) every form

Every form carries a header block the tool should extract:

Field	Notes
Participant's Name	<i>Personal health info — see §10 privacy note</i>
Participant's Age	Some categories are adults-only (18+)
FI Coordinator Name	
Broker Name	
Requested item/class/service/program/membership name	Label varies by form
Provider / Vendor name	
Link to webpage / place of publication / item link	The URL the tool must research
Fee / rate / price	e.g. "Fee per Session", "Membership Fee", "Fee per Course", "Fee per Class"
Valued outcome + Life-Plan date	Internal — copied from the participant's care plan; not website-verifiable
YES/NO checklist questions	The category-specific criteria (see §5)
Signatures + "For office use only" (Approve/Not Approved, Ref #)	The tool does not fill these in

4.2 The seven forms

1. **Community Classes** – the primary, highest-volume case. Also has an **Appeals** variant (§7).
2. **Coaching (for Parents/Spouse)** – educational courses; adults 18+; capped rates.

3. **Health-Club / Organizational Memberships** – gym/organization memberships.
 4. **Household Related Items (HRI)** – equipment/items for the home (product link, not a class site).
 5. **OTPS (Other Than Personal Services)** – miscellaneous goods/services; has an exclusion list.
 6. **Transition Program** – vocational/skill programs for adults who aged out of school.
 7. **Pre-Approval Appeals** – submitted after a denial; includes the denial reason and a justification, and asks the tool to re-examine the website evidence in light of that reason.
-

5. The checklist logic (the most important clarification)

Each form has a YES/NO checklist. **The tool cannot answer all of it from a website.** Many items depend on internal data the tool doesn't have — whether the category is *approved in the participant's budget*, whether it matches a goal in the participant's *Life Plan*, whether it duplicates other funded services, etc. **The tool's job is only the website-verifiable subset**, plus a fee/cap sanity-check. Everything else it should mark "**Internal — not website-verifiable**" and leave for the human.

Below is the website-verifiable subset per category. (The relevant program caps are included so the tool can flag when a stated fee exceeds the program limit.)

Community Classes — website-verifiable

- Class is **open to and attended by the broader public** (not restricted to disabled/OPWDD individuals)
- Class has **published fees** (a public price is visible)
- Fees are **identical for OPWDD and non-OPWDD** individuals (no special/segregated pricing)
- Class is **subject-based** (art, dance, martial arts, cooking, etc.)
- Class does **not** give **college credits**
- Class is **not clinical/therapy** in nature
- A **published schedule** exists
- **Fee-match:** published fee is consistent with the "Fee per Session" on the form
- *Invoice-rule context:* >2 sessions/day (≤60 min each) or >7 sessions/week at one provider needs prior approval

Coaching — website-verifiable

- **Published fees** exist for the course/class
- Fee sanity-check: ≤ **\$55 per group class** / ≤ **\$111 per private class**, program **capped \$500/year**
- Content is a genuine **educational/coaching opportunity** (not lodging/travel, which is excluded)

Memberships — website-verifiable

- Organization is **open to the public** — **not** a private/invitation-only club
- **Membership fee is published**; check the published amount against the form's fee (yearly/monthly)

HRI (Household Related Items) — website-verifiable

- The **item exists at the linked page**, with a visible **price**
- Item plausibly **not on the exclusion list** (e.g., not a cell phone, computer, vehicle, medical device, pill dispenser)
- **Safety features** described on the form are supported by the product page
- Cap context: **\$1,500 / budget year**; approval valid **3 months**

OTPS (Other Than Personal Services) — website-verifiable

- The **item exists at the linked page**, with a visible **price**
- Item plausibly **not on the OTPS exclusion list** (e.g., not cable TV, common household supplies, rental cars, legal fees, co-pays, experimental therapies)
- Cap context: **\$3,000 / budget year**

Transition Program — website-verifiable

- **Published fees** exist
- Fee sanity-check: $\leq \$350$ per course / $\leq \$800$ per month
- Program is **noncredit-bearing** and oriented to **skill-building / employment outcomes**
- Program is **not** in an **OPWDD-certified location**
- (Staff background-screening is proven by a letter, not the website — flag as "needs document")

Appeals — website-verifiable

- Re-run the **Community Classes** website checks, **framed against the stated denial reason**, and surface evidence that specifically supports or refutes that reason.

Design implication: the mapping "form → checklist → which items are website-verifiable → how to check each" is configuration/knowledge the system applies. Structure it so a non-engineer could add a new category later by supplying a new checklist — don't hard-code it into the control flow.

6. Output — the report

For each application the tool produces a **shareable report** plus its **evidence captures**, bundled so a reviewer can save and share them.

We are intentionally not prescribing the report's layout. *How* you present the report is part of what we're evaluating — show us your judgment. What matters is that it clearly **communicates** the items below.

The report must convey:

- **The request, at a glance** — participant name; provider/vendor; the specific class/item/service/program/membership; the website/link; the category; and the date the review was done.
- **A rate comparison** — the **rate requested in the application** vs. the **rate found on the website**, with a plain-language verdict (e.g. "*matches application exactly*," "*differs*," "*not published*").
- **Per-criterion findings** — for each website-verifiable checklist item: its **status** (met / not met / needs review — your wording), the **evidence URL**, a reference to the **screenshot/evidence**, and a **plain-language note** saying what the page shows (ideally quoting the relevant line).
- **Items the website can't answer** — clearly marked as internal / not verifiable online, **not guessed**.
- **The evidence captures** — see below.

Evidence captures — for the audit file, the tool produces **two kinds** of evidence:

1. **A whole-page capture of the provider's website** — one **screenshot_or_PDF** of the **entire page** (the full homepage, and/or the specific page carrying the proof such as the fees or schedule page), top-to-bottom, so the full context is preserved. This is the "here is the website we reviewed" record.
2. **Targeted evidence captures, one per confirmed requirement** — a focused screenshot showing exactly where the proof appears, **labeled with the requirement it supports** (e.g. "Evidence: published fees," "Evidence: open to the public"), as in a normal evidence file.

Both kinds must be date-stamped with a visible **date/time of when the capture was taken** (an audit needs to prove what the site showed on that date) and must record the **URL** captured.

7. User interaction

The reviewer should be able to work with the tool in **plain language**, e.g.:

- "Review this application." · "Use the website listed on the form."
- "Check whether the requested class and its rate are listed."
- "Change this item to Needs Review." · "Add a note to the report." · "Regenerate the report."

The tool should **ask for clarification** when key info is missing or ambiguous: no URL on the form, unclear provider/vendor, unclear requested item, missing/ambiguous category, or more than one plausible checklist.

8. Acceptance criteria — a successful basic version

1. Accepts a completed pre-approval application (one of the 7 forms).
2. Extracts the main request details (§4).
3. Identifies the website/link to review.
4. Identifies the applicable category.
5. Applies the **correct checklist** and correctly separates **website-verifiable** items from **internal** ones (§5).
6. Visits the website and gathers **public evidence** for the website-verifiable items.

7. Captures **date-stamped evidence** for what it found — a **whole-page** capture of the site (screenshot or PDF) **plus** a labeled per-requirement screenshot for each confirmed item — each stamped with the capture date/time and URL.
8. Marks items **Not Found / Needs Review** when evidence is absent or unclear.
9. Generates a **readable report** (§6) and summarizes results for the reviewer.
10. Lets the reviewer request **basic changes** to the report.

Always: the tool supports review and evidence-gathering; **final approve/deny stays with staff.**

9. Deliverables (what each candidate hands back)

Each candidate submits **three things**:

1. **The finished product** — a working tool that runs the workflow above (CLI, web UI, or chat interface; your choice, justify it). It should run on **at least three** of the provided samples and produce the report package (§6) for each.
2. **A Git repository** of their work — organized the way you'd want a team to inherit it: README, clear structure, config-driven checklists, and tests where appropriate. **No real participant data committed.**
3. **A short video** walking through how they approached the problem and **which tools/models they used.**

Test data provided: ten **completed sample application forms** accompany this brief (populated versions of the real forms, one PDF each — see *Pre-Approval Tool — Sample Applications*). They use **fictional participants** but **real, public provider URLs**, and deliberately span every form type plus a spread of difficulty: clearly-published evidence, ambiguous/gated pricing, an exclusion-list item that should be flagged, and an appeal.

The product and repo should also include: **simple run instructions** for a non-technical reviewer; at least one **sample output report** (with the evidence captures); a short note on **how to add or change a form/checklist**; and a statement of **limitations, assumptions, and what production-readiness would require.**

10. Constraints, non-goals & data handling

- **Human-in-the-loop, always.** The tool never issues an approval or denial. It gathers evidence and flags; a reviewer decides.
- **Privacy / sensitive data (important).** Real applications contain participant names, ages, and other **personal health information (PHI)**. For this exercise, use only **redacted or synthetic sample data** — do **not** send real participant records to any third-party service, and do **not** commit any real data to your repo. Assume a production version would run under strict privacy controls; note in your write-up how your design would handle that.

- **Websites vary wildly.** Provider sites are inconsistent — some have no published fees, some are PDFs/flyers, some are social-media pages. Handle "evidence not found" gracefully; don't fabricate. Honest "Not Found / Needs Review" is a **correct** result.
 - **No guessing on the checklist.** If something can't be verified from the public web, mark it Internal or Needs Review — don't infer a YES/NO.
 - **Not in scope:** integrating with our internal systems (Enterprise Manager, eVero), the participant's budget, or the Life Plan; making the actual approval decision; handling payment/invoicing.
-

11. How we'll evaluate you

We care less about a polished UI than about how you think. We'll look at:

- **Problem framing** — did you correctly separate "what a website can prove" from "what it can't"? (This is the crux — see §5.)
 - **Extraction quality** — robustness reading real, imperfect, scanned forms.
 - **Research/agent design** — how the tool navigates a site, decides it's "found" the evidence, and knows when to stop or ask.
 - **Evidence integrity** — screenshots and citations that would actually hold up in an audit; no hallucinated findings.
 - **Report clarity** — could a non-technical reviewer act on it immediately?
 - **Repo quality** — structure, README, config-driven checklists, sensible use of AI models/tools, and honest handling of limits.
 - **Judgment on the AI stack** — your model/tooling choices and *why*.
-

12. Glossary

Term	Plain meaning
OPWDD	NY State Office for People With Developmental Disabilities — funds and regulates these services
Medicaid	Government health program; ultimately funds the budget and audits spending
Self-Direction (SD)	Program where the participant controls their own government-funded care budget
Participant	The person with a developmental disability receiving services
FI (Fiscal Intermediary)	The agency that holds the budget, employs staff, and enforces the rules
FI Coordinator	Agency staffer managing a caseload of participants
FI intake	Onboarding a new participant into the program (not the focus of this project)
Pre-approval	Required sign-off before spending budget money on non-routine items — the focus of this project
Pre-Approvals team / Reviewer	Agency staff who review pre-approval applications; the tool's user
Budget	The participant's government-funded spending plan
Life Plan (LP)	The participant's official care plan; lists goals and "valued outcomes" (internal, not on any website)
Broker / Support Broker	Independent professional who helps write the participant's plan
Care Manager / CCO	The managed-care organization that coordinates the participant's services
IDGS	"Individualized Directed Goods and Services" — the budget category many of these requests draw from
OTPS	"Other Than Personal Services" — miscellaneous goods/services budget category
HRI	Household Related Items
Valued outcome	A goal from the participant's Life Plan the purchase is supposed to support

Appendix A — Source forms provided

Community Classes Preapproval Form.pdf , Coaching Preapproval Form.pdf , Memberships preapproval form.pdf , HRI Justification form.pdf , OTPS Justification Form.pdf , Transition Program Preapproval Form.pdf , Preapproval Appeals Form.pdf

(A blank template shows the header fields and the full YES/NO checklist for each category. A real application is one of these, filled in — usually a scan/PDF.)